跨项目智能三方研究综合报告

日期：2026-03-15

三方：Claude Opus（知识工程+产品架构）/ Gemini（产品与用户价值）/ Codex（工程实现与算法）

一、三方共识（独立得出相同结论的高置信度发现）

共识 1：这是范式转变，不是渐进演化

三方独立得出同一判断：

- Claude："价值主张变了、竞争壁垒变了、产品边界变了。"
- Gemini："从工具到平台的范式转变"，"从'帮我读完这个项目'到'告诉我这个领域的真相'"
- Codex："Doramagic 已经不再只是单项目灵魂提取器，而是在逼近一种'项目间知识关系引擎'"

结论：跨项目智能不是灵魂提取的附加功能，而是 Doramagic 的第二条增长曲线。

共识 2：知识需要三层表示才能可比较

- Claude：知识天然形成三层（共识层 / 架构层 / 独创层）
- Codex：指纹必须三层（Code Fingerprint / Knowledge Atom / Soul Graph），因为 fork 检测看代码、公约数看原子知识、设计哲学看图
- Gemini：竞品比较暗示三层（npm trends 比元数据、GitHub 比代码、只有 Doramagic 比知识）

结论：单层表示（如纯 embedding 或纯文本 diff）无法胜任跨项目对比。必须分层。

共识 3："假公约数"是核心风险

- Claude：提出独立性评分（project_independence × evidence_diversity × source_independence），以 Ahrefs 0.737 为案例说明"4 个项目引用同一研究 ≠ 4 次独立验证"
- Codex："同样的知识被多个项目提到，不代表它被多次独立验证"，提出 provenance clustering
- Gemini：间接触及——"每个作者都有自己的偏见，但 4 个项目里有 3 个提到的点，大概率是'客观真理'"（注：Claude 和 Codex 反驳了这个朴素假设）

结论：公约数的可信度取决于来源独立性，不是覆盖数量。`support_independence` 是公约数系统的生死线。

共识 4：不要给出武断标签，要给概率/多维度输出

- Claude：SoulDiff 输出 5 种关系类型 + 共享/独创/冲突知识 + 推荐
- Codex：输出概率分布（fork_likelihood / wrapper_likelihood / convergent_evolution_likelihood / shared_upstream_pattern_likelihood）
- Gemini：交互设计用韦恩图展示重叠而非二分判断

结论：Doramagic 应遵循"不教用户做事"的产品哲学——给出 X 光片，不给出诊断结论。

共识 5：时间维度是核心特征而非辅助特征

• Claude：temporal_validity（introduced/deprecated/last_confirmed）+ 知识生命周期追踪

• Codex："时间线不是辅助特征，而是核心特征。没有时间，就没有 lineage。" atom 第一版就要带 first_seen/last_verified/applies_to_version

• Gemini：时间轴溯源作为交互设计核心——展示知识最早出现在哪个 commit

结论：从第一天就把时间戳建进数据结构。没有时间的知识指纹是残缺的。

二、各方独有贡献（单方最佳洞察）

Claude 独有：战略级洞察

1. 权重冲突 = 领域路线之争：claude-seo 把技术 SEO 权重定为 25%，geo-seo-claude 定为 15%。这不是"谁对谁错"，而是 2026 年 SEO→GEO 范式转移的直接反映。"冲突本身是高价值知识"——这是一个新的认知。

2. 第二飞轮：跨项目知识图谱的网络效应（提取越多 → 公约数越可靠 → 吸引同领域用户 → 更多项目被提取）。建议"按领域深耕"而非"广撒网"。

3. 好作业评判公式修正：增加 NetworkValue 维度（consensus_reinforcement × novelty_contribution × conflict_informativeness × domain_coverage_gap）。

4. 知识供应链分析：Doramagic 是唯一能看到整个知识供应链的角色——上游源头（Ahrefs/Princeton/Google）→ 中游项目（提取对象）→ 下游消费者。可以检测传播扭曲和单点依赖。

5. 反向公约数（异常检测）：只有 1 个项目提到的知识 → 分类为 SUSPECT / EMERGING / NICHE / UNIQUE_INSIGHT。

6. 领域成熟度指标：公约数比例 >70% = 成熟领域；<30% = 新兴领域；中等但冲突多 = 范式转移中。SEO/GEO 属于"TRANSITIONAL（公约数 45%，冲突 15%，独创 40%）"。

7. 盲区指出：跨项目对比应优先用 Stage 0-3 结构化产物，不是 Stage 4 叙事。叙事是给人看的，对比要用原始数据。

Gemini 独有：产品与用户视角

1. 两类用户故事的精准定义：

• "项目打架" = 选型者的"排雷"与"溯源"（效率焦虑 + 可靠性恐惧）

• "最大公约数" = 学习者的"行业标准说明书"（寻找共识 + 过滤噪音）

2. 市场叙事升级：从"帮你偷懒"到"定义什么是好、什么是真"。用户心智从"帮我读完这个项目"变成"告诉我这个领域的真相"。

3. 竞品差异化定位（非常清晰）：

• npm trends 比"面子"（元数据），Doramagic 比"里子"（设计意图）

• GitHub Compare 告诉你代码改了哪一行，Doramagic 告诉你灵魂有 80% 重合

• Sourcegraph 是搜索引擎，Doramagic 是分析报告生成器

4. 非技术用户价值：

• 投资人："这个创业项目核心灵魂与 3 个开源项目 95% 重合，属于套壳，慎重投资"

• 产品经理："竞品都通过 A 方式解决 B 问题，我们快速输出基础功能，专注独创部分"

• 决策者："技术纯度报告"

5. 三个脑洞：

• "灵魂撞衫"报警器（发布新项目时自动检测与已有项目重合度）

• "知识通胀"检测（监控某知识从前沿变成基础设施的过程）

• 自动"缝合怪"生成（取各项目最佳部分自动组合）

6. 交互设计："Knowledge Lab" — 多选入场 → 韦恩图碰撞 → 时间轴溯源

Codex 独有：工程深度

1. 三层指纹的具体数据结构：

• `project_fingerprint.json` 完整 schema（metadata + code_fingerprint + knowledge_atoms + soul_graph）

• Knowledge Atom 的 canonicalization：subject/predicate/object/modifiers slot 结构，支持精确 match + 语义 match + slot-aware match

• Soul Graph 的最小 schema：6 种节点类型（principle/module/workflow/constraint/pitfall/artifact）+ 6 种边类型（motivates/implemented_by/guards/depends_on/conflicts_with/specializes）

2. 公约数的三层匹配算法（最具工程可行性）：

• 第 1 层 Lexical overlap：MinHash / SimHash / shingles → 抓套壳和轻度改写

• 第 2 层 Semantic overlap：embedding + FAISS/HNSW ANN → 抓同义不同表述

• 第 3 层 Structured overlap：typed slot matching → 过滤"语义接近但作用域不同"的假共识

3. Fork 检测的正确分工：代码层用于 lineage 候选生成，知识层用于解释相似性的性质。不能单靠任何一层。

4. $O(N^2)$ 优化策略：metadata bucketing → LSH/ANN 近邻召回 → project families 聚类 → 桶内精对比。"不要一上来做 graph matching。图匹配最贵，应该只在少量候选上跑。"

5. 可复用工具清单：

• datasketch MinHash LSH（集合相似候选生成）

• Tree-sitter（统一 AST 解析）

• JPlag（程序相似检测）

• FAISS（高维向量 ANN）

• OpenEA（KG alignment 思路，但不要直接搬框架——LLM 提取的图噪声大）

6. MVP 路径（最清晰的实施建议）：

• 第一步：新增 `project_fingerprint.json`（现有输出旁边多一个编译产物）

• 第二步：`compare_projects.py`（独立脚本，输入多个 fingerprint.json → 输出 similarity/lineage/consensus/unique）

• 第三步：生成 `CROSS_PROJECT_REPORT.md` / `CONSENSUS.md` / `SOUL_DIFF.md`

7. 四维评分体系：Similarity Score / Lineage Score / Consensus Score / Uniqueness Score

8. 关键工程挑战：

• scope normalization 比 embedding 更重要（"FAQPage 对 Google rich results 被限制" vs "FAQPage 对 AI crawler extraction 仍有价值"——没有 scope 区分就会被错误合并）

• 评估数据集会成为真正门槛（需要 3 类 gold set：已知 fork pairs / 已知趋同 pairs / 已知领域共识 atoms）

• 时间版本必须从第一天加

三、三方分歧与决策建议

分歧 1：公约数的可信度模型

• Claude：ConsensusConfidence = f(coverage_ratio, independence_score, evidence_diversity, temporal_consistency, specificity_agreement)

• Codex：Consensus = support_count × support_independence × semantic_agreement × scope_compatibility

• Gemini：未提出具体模型，倾向于用覆盖数量作为可信度

决策建议：Claude 和 Codex 的模型高度互补。采用 Codex 的四因子公式作为核心，加入 Claude 的 evidence_diversity 和 temporal_consistency 作为补充维度。Gemini 的朴素覆盖数量假设被 Claude 和 Codex 共同反驳——不应作为单独指标。

分歧 2：输出格式

• Claude：SoulDiff 结构（relationship enum + shared/a_only/b_only/conflicts + recommendation）

• Codex：概率分布（fork_likelihood / wrapper_likelihood / convergent_evolution_likelihood / shared_upstream_pattern_likelihood）

• Gemini：韦恩图 + 时间轴

决策建议：三者不矛盾，服务不同消费者。

• 概率分布（Codex）→ 机器消费和 API

• SoulDiff 结构（Claude）→ 程序化报告

• 韦恩图 + 时间轴（Gemini）→ 用户界面

分层实现：先做 Codex 的 fingerprint + scores，上面构建 Claude 的 SoulDiff，最后 Gemini 的 UI 渲染 SoulDiff。

分歧 3：实施优先级

• Claude：P0 = KnowledgeClaim 数据结构 + 灵魂 diff → P1 = 独立性评分 + 公约数 → P2 = 时效追踪 + 领域成熟度

• Codex：P0 = project_fingerprint.json → P1 = compare_projects.py → P2 = 报告生成

• Gemini：先做"Knowledge Lab"交互原型

决策建议：采用 Codex 的实施路径（最具工程可行性），用 Claude 的数据结构设计填充内容。Gemini 的交互设计留到有可工作的 API 之后。

四、Brief 未提到、三方发现的新可能性

可能性	提出方	简述
知识时效追踪	Claude + Codex	跨项目自动检测过时知识（单项目不知道自己过时，领域图谱能追踪）
知识供应链分析	Claude	追踪知识从上游源头 → 中游项目 → 下游消费者的传播路径，检测扭曲和单点依赖
反向公约数（异常检测）	Claude	只有 1 个项目提到的知识分类为 SUSPECT/EMERGING/NICHE/UNIQUE_INSIGHT
领域成熟度指标	Claude	公约数比例 = 领域成熟度的元指标
"灵魂撞衫"报警器	Gemini	发布新项目时自动检测重合度
"知识通胀"检测	Gemini	监控知识从前沿变成基础设施
自动"缝合怪"生成	Gemini	从 N 个项目取各最佳部分自动组合
知识传播图	Codex	atom-level first-seen timestamp → 不只是相似性引擎，变成知识传播分析器
错误共识检测	Codex	provenance clustering → 识别"一个错误被多次传播"
领域共识图谱	Claude + Codex	多项目公约数积累 → 领域级知识地图

五、最终行动计划

阶段 0：数据结构（1-2 天）

- 在现有 Soul Extractor 输出旁新增 `project_fingerprint.json`
- Schema 包含：metadata + code_fingerprint（路径骨架 + 命令集 + 依赖集）+ knowledge_atoms（canonical claim + embedding + scope + temporal）+ soul_graph（typed nodes + edges）
- 来源：Stage 0 → code_fingerprint；Stage 1-3 → atoms + graph；不用 Stage 4 叙事

阶段 1：对比脚本（2-3 天）

- `compare_projects.py`：输入多个 fingerprint.json → 输出 4 维评分（Similarity / Lineage / Consensus / Uniqueness）
- 三层匹配：lexical（MinHash）→ semantic（embedding ANN）→ structured（slot matching）
- 独立性评分：排除 fork/同源，追溯上游源头

阶段 2：报告生成（1-2 天）

- `SOUL_DIFF.md`：两两对比，输出 SoulDiff 结构
- `CONSENSUS.md`：N 项目公约数，标注独立性和时效性
- `CROSS_PROJECT_REPORT.md`：综合分析（关系图 + 去重 + 公约数 + 独创 + 推荐 + 路线分歧）

阶段 3：验证（1 天）

- 用 exp-seo-skills 的 4 个项目作为 gold set
- 验证：claude-seo ≈ 30x-seo 是否被正确识别为高 lineage
- 验证：共享知识（FAQPage 等）是否被正确提取为公约数
- 验证：独创知识（citability scorer、context 底板）是否被正确标注

后续：

- 领域深耕策略（先在 SEO/GEO 建密集图谱）
- 知识时效追踪
- 评估数据集构建（3 类 gold set）
- 交互设计（Knowledge Lab）

六、一句话总结

> 单个灵魂是知识。多个灵魂的关系是智慧。Doramagic
> 从"项目阅读工具"进化为"开源领域知识基础设施"——这不是功能扩展，是范式转变。